

```
#line 7 "eg3.lex"
{ charCount++; }
YY_BREAK
case 2:
YY_RULE_SETUP
#line 8 "eg3.lex"
ECHO;
YY_BREAK
```

This is generated corresponding to the rule section in the sample program. We can see that case 0 is for input retraction. This is needed because it may be that only after reading the next character does the compiler understand that the character read is part of the next token. In that case, the file pointer needs to be retracted. Case 1 includes the action specified in Rule 1, to increment the value of the variable `charCount`. Case 2 is for simply printing the unrecognised symbol to the output stream. `ECHO`, seen in Case 2, is a macro name and its definition is as follows:

```
#define ECHO do { if (fwrite( yytext, yyleng, 1, yyout )) {} }
while (0)
```

`ECHO` will simply write the content of `yytext` to the output stream. The use of `yytext` is mentioned in the next section.

The definition of `YY_BREAK`, a macro name seen in Case 2 of the above switch case block, is as follows:

```
/* Code executed at the end of each rule. */
#ifdef YY_BREAK
#define YY_BREAK break;
#endif
```

Thus `YY_BREAK` is to break the flow of execution.

Built-in functions and variables

The Lex compiler provides us with a number of built-in variables and functions. Some of them are listed below.

- The Lex library contains the definition of `main()` function. Programmers can either simply include `main()` from the library or can define their own `main()`. The `main()` function defined in the library contains a call to the function `yylex()`. The default definition of `main()` is as follows:

```
main(int ac, char **av)
{
    return yylex( ) ;
}
```

The variable `yytext` is a character array that contains a character/group of characters read from the input file for matching with any pattern specified in the rule section.

- The function `yywrap()` returns 1 if EOF is encountered, otherwise 0 is returned. One can modify this function according to one's needs so that it may return 1 until all the

files required are read.

- The variable `yylen` is the length of string in `yytext`.
- Some alternatives of Lex define the variable `yylineno`. It keeps track of the line number which is required for error reporting. `yylineno` is updated when the pattern for the new line (`ln`) is matched.

Sample program 2

The following program will make it clear how users can define their own sub-routines to use in a Lex program.

```
%
[0-9]+ {display(yytext);}
. {}
%
display(char a[])
{
    printf("%s\n",a);
}
```

The above program prints nothing but the combination of digits in the input to the output stream. We can see that the second pattern specified in the rule section is a simple `.` (dot) and the action part specified as `{}`. This means whenever this pattern matches, do nothing. You can see the variable `yytext`, which contains the current lexeme, is passed as the argument to the user defined function `display()`. Another point to be noted is that this program does not contain a `main()` function. The default definition of `main()` must be included while this program is compiled. Otherwise an error will be generated. As stated earlier, the definition of `main()` can be included by compiling `lex.yy.c` with the `lfl` or `ll` options.

Lex is a very powerful tool that can be used to manipulate text that involves patterns. It gains its power from regular expressions. Luckily, the Lex language is compatible with C, a very commonly used programming language known to even high school students and thus, many of the problems solved using C can be coded in Lex. This simple tool makes the job of programmers easier.

We cannot downgrade the role of Lex to that of merely a simple pattern matcher. It finds application in the lexical analysis phase of a compiler too. More details about the construction of a lexical analyser using Lex can be seen in the book 'Compilers: Principles, Techniques and Tools' by Alfred V. Aho, Ravi Sethi and Jeffrey D. Ullman. 

References

- <http://dinosaur.compilertools.net/lex/>
- [https://en.m.wikipedia.org/wiki/Lex_\(software\)](https://en.m.wikipedia.org/wiki/Lex_(software))

By: Sharon Sunny

The author is an assistant professor at Amaljiyothi College of Engineering, Kerala. He can be reached at ssharon099@gmail.com.